



genpact

Salient Object Detection Using Convolutional Encoder-Decoder Networks

Written by: Dea Ceku

1. Introduction

1.1 Background

Salient Object Detection (SOD) is a task in computer vision that focuses on finding and separating the main objects in an image that naturally draw human attention. The goal is to generate a binary mask that distinguishes important foreground objects from the background. This is useful in many areas of computer vision, including image segmentation, object tracking, and robotics.

Applications of SOD include:

- **Image Processing:** Used for tasks such as foreground extraction, smart cropping, and image resizing.
- **Computer Vision:** Helps improve object tracking, recognition, and classification systems.
- **Assistive Technology:** Can support visually impaired users by identifying key objects in a scene.

This project is important because Salient Object Detection is a practical computer vision task that connects image segmentation, deep learning, and real-world visual understanding. By building the full pipeline from scratch, the project helps demonstrate how images and masks are prepared, how a CNN encoder-decoder model learns spatial features, and how model performance can be measured using segmentation metrics. It also provides experience with training, debugging, improving, and evaluating a deep learning model in a complete end-to-end workflow.

1.2 Project Objective

The objective of this project is to design and implement a Salient Object Detection system using a convolutional encoder-decoder neural network. The model is trained to take an input image and generate a saliency mask that highlights the most visually important object or region in the image.

This project focuses on building the full deep learning pipeline from scratch, including dataset preparation, preprocessing, model design, training, evaluation, and visualization. A baseline CNN model is first developed, then an improved version is created using techniques such as Batch Normalization, Dropout, and increased feature channels.

The goal is to evaluate how well the model can detect salient regions by using metrics such as Intersection-over-Union, Precision, Recall, and F1-score. The project also aims to demonstrate how architectural improvements can affect segmentation performance.

2. Dataset & Preprocessing

2.1 Dataset

The DUTS dataset was used for training and evaluating the Salient Object Detection model in this project. DUTS is a large-scale benchmark dataset specifically designed for saliency detection tasks and contains both training and testing image-mask pairs. The dataset includes 10,553 training images and 5,019 testing images collected from the ImageNet DET dataset and the SUN dataset.

Each image is paired with a pixel-level ground-truth saliency mask that highlights the visually important object or region within the image. The annotations were manually created by multiple human subjects to ensure accurate segmentation quality. The dataset contains a wide variety of scenes and challenging object structures, making it suitable for evaluating deep learning-based segmentation models.

The dataset used in this project was obtained from Kaggle:

<https://www.kaggle.com/datasets/balraj98/duts-saliency-detection-dataset>

2.2 Data Splitting

The dataset was divided into three subsets in order to train and evaluate the model effectively. The training set contained 70% of the images and was used for learning the model parameters. The remaining data was split equally into validation and testing sets, each containing 15% of the dataset. The validation set was used to monitor the training process and reduce overfitting, while the test set was used for final performance evaluation.

2.3 Preprocessing

Several preprocessing steps were applied before training the model. All input images and masks were resized to a fixed resolution of 128×128 pixels to maintain consistency throughout the dataset and reduce computational complexity. Pixel values were normalized to the range between 0 and 1, which helps stabilize the learning process and improves convergence during training. Finally, the images and masks were converted into PyTorch tensors so they could be efficiently processed by the neural network.

Example Preprocessing Snippet

```
image = cv2.resize(image, (128, 128))
image = image / 255.0

image = torch.tensor(
    image,
    dtype=torch.float32
).permute(2, 0, 1)
```

The images were resized to 128×128 pixels and normalized to the range [0,1]. The processed images were then converted into PyTorch tensors before being passed to the neural network.

2.4 Data Augmentation

Data augmentation techniques were applied to the training images in order to improve the model's ability to generalize to unseen data and reduce overfitting. Since deep learning models can become sensitive to limited image variations, augmentation helps increase the diversity of the training dataset without collecting additional images.

In this project, random horizontal flipping and brightness variation were used as augmentation methods. Horizontal flipping helps the model learn object patterns from different orientations, while brightness adjustment improves robustness against different lighting conditions.

The augmentation operations were applied randomly during training so that the model could observe slightly different versions of the same image during each epoch.

Example augmentation snippet:

```
# Random horizontal flip
if random.random() > 0.5:
    image = cv2.flip(image, 1)
    mask = cv2.flip(mask, 1)

# Random brightness adjustment
if random.random() > 0.5:
    brightness = random.uniform(0.7, 1.3)
    image = np.clip(image * brightness, 0, 255).astype(np.uint8)
```

3. Model Architecture

3.1 Baseline CNN

The baseline model used in this project follows an encoder-decoder Convolutional Neural Network architecture designed for image segmentation tasks. The encoder is responsible for extracting important spatial features from the input image, while the decoder reconstructs these learned features into a saliency mask.

The encoder section consists of multiple Conv2D layers followed by ReLU activation functions and MaxPooling operations. The convolutional layers learn image features such as edges, textures, and object structures, while MaxPooling reduces the spatial dimensions and helps the network focus on the most important features.

The decoder section uses ConvTranspose2D layers to progressively upsample the feature maps back to the original image resolution. A final Sigmoid activation function is used to generate a one-channel saliency mask with pixel values between 0 and 1.

3.2 Improved CNN

After building the baseline model, an improved version of the architecture was developed to enhance segmentation performance and training stability. Several modifications were introduced, including Batch Normalization, Dropout regularization, and increased feature channel sizes.

Batch Normalization layers were added after convolutional operations to stabilize the learning process and accelerate convergence during training. Dropout was introduced in the deeper encoder layers to reduce overfitting by randomly disabling neurons during training. Additionally, the number of feature channels was increased to allow the network to learn more detailed spatial representations from the input images.

3.3 Loss Function

The model was trained using a combined loss function consisting of Binary Cross Entropy (BCE) loss and Intersection-over-Union (IoU) loss.

$$\text{Loss} = \text{BCE} + 0.5(1 - \text{IoU})$$

Binary Cross Entropy measures pixel-wise classification accuracy by comparing predicted mask values with the ground-truth masks. However, BCE alone may not fully optimize segmentation overlap quality. To address this, IoU loss was added to improve the overlap between predicted salient regions and the ground-truth masks.

By combining both losses, the model learns not only accurate pixel predictions but also better object-level segmentation consistency.

4. Training and Evaluation

4.1 Training Configuration

The models in this project were trained using the Adam optimization algorithm, which is widely used in deep learning because of its efficient convergence and adaptive learning capabilities. A learning rate of 0.001 was selected to balance training stability and convergence speed.

The training process was performed for up to 25 epochs with a batch size of 8 images per iteration. During training, both training loss and validation loss were monitored after each epoch in order to evaluate the learning progress of the model.

Early stopping was also implemented to reduce overfitting and prevent unnecessary training once the validation loss stopped improving. In addition, the best-performing model weights were automatically saved during training for later evaluation and inference.

4.2 Evaluation Metrics

Several evaluation metrics were used to measure the performance of the Salient Object Detection model. Since segmentation tasks require accurate pixel-level predictions, multiple metrics were necessary to evaluate different aspects of model performance.

Intersection-over-Union (IoU)

- Intersection-over-Union measures the overlap between the predicted saliency mask and the ground-truth mask. Higher IoU values indicate better segmentation accuracy and stronger overlap consistency between predictions and target regions.

Precision

- Precision measures how many predicted salient pixels were actually correct. High precision indicates that the model produces fewer false positive predictions.

Recall

- Recall measures how many ground-truth salient pixels were successfully detected by the model. High recall indicates that the model is able to capture more relevant salient regions.

F1-Score

- The F1-score combines Precision and Recall into a single metric and provides a balanced evaluation of segmentation quality. It is especially useful when comparing models with different prediction behaviors.

5. Results & Discussion

5.1 Quantitative Results

Both the baseline CNN model and the improved CNN model were evaluated using multiple segmentation metrics, including Intersection-over-Union (IoU), Precision, Recall, and F1-score. The evaluation was performed on the test dataset after training was completed.

The quantitative results are presented in Table 1.

Model	IoU	Precision	Recall	F1 Score
Baseline CNN	0.4704	0.6235	0.6938	0.6077
Improved CNN	0.5009	0.6702	0.6932	0.6349

The improved CNN achieved higher performance across most evaluation metrics. In particular, the IoU score increased from 0.4704 to 0.5009, indicating better overlap between the predicted saliency masks and the ground-truth masks. Precision and F1-score also improved, suggesting that the improved architecture generated more accurate and stable segmentation outputs.

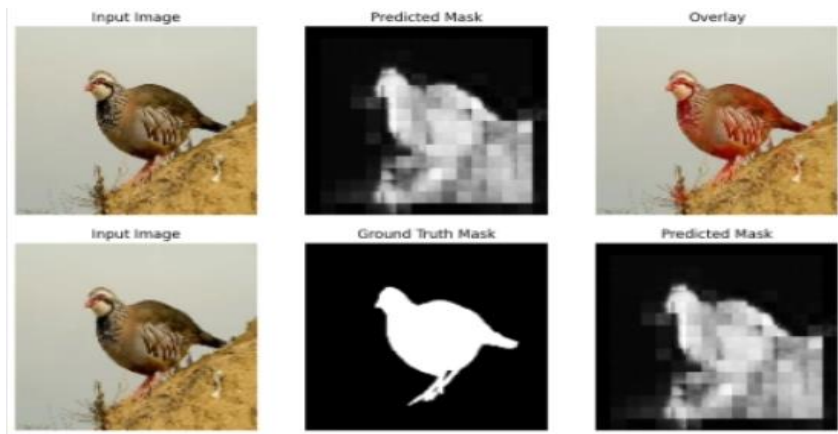
5.2 Visual Results

Visual inspection of the generated saliency masks showed that both models were capable of learning the general shape and location of salient objects within the images.

Figure 1. training vs validation loss graph



Figure 2. Prediction visualization image



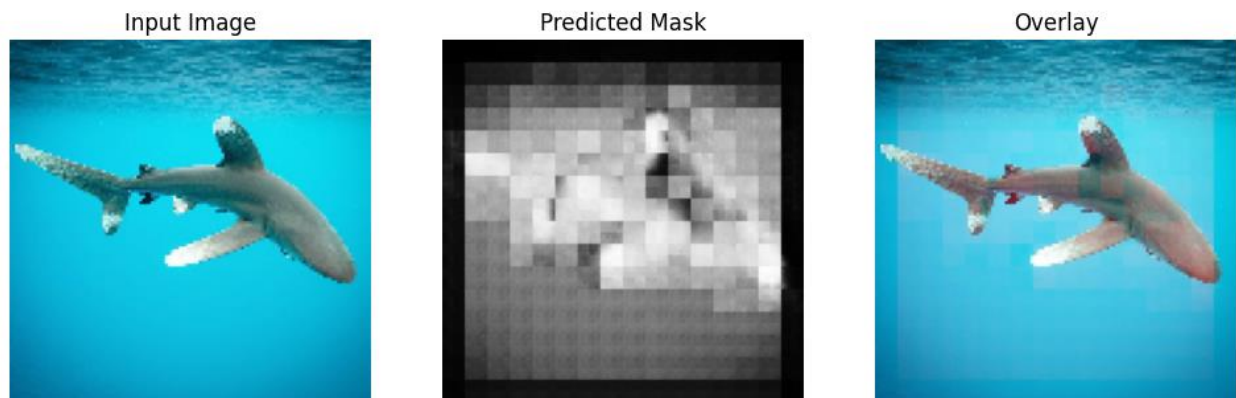
The predicted masks successfully highlighted the dominant object regions while suppressing most background information.

The improved CNN produced cleaner segmentation masks with better object boundaries and reduced noise compared to the baseline architecture. Overlay visualizations further demonstrated that the improved model was more effective at capturing the full salient object region and preserving important structural details.

Although the generated masks were not perfectly accurate in all cases, the overall segmentation quality demonstrated that the encoder-decoder architecture was able to learn meaningful spatial representations directly from the training data.

Figure 3. Real-time inference demo using a custom uploaded image.

The demo allows users to upload external images and generate saliency masks together with overlay visualizations and inference timing.



The predicted masks successfully highlighted the primary object regions in most images, although some noise and blurry boundaries were still present in the segmentation outputs.

The overlay visualizations further demonstrated that the model was able to focus on the dominant object regions while suppressing most of the image background. However, some fine details and object

boundaries were not perfectly segmented due to the relatively simple encoder-decoder architecture and low image resolution.

5.3 Discussion

The results show that the improved CNN architecture achieved better segmentation performance than the baseline model. The addition of Batch Normalization layers helped stabilize the training process and improved feature learning, while Dropout regularization reduced overfitting by preventing the network from relying too heavily on specific neurons during training.

The use of the combined BCE + IoU loss function also contributed to improved segmentation quality by optimizing both pixel-level classification accuracy and region overlap consistency.

Despite these improvements, some limitations were still observed. Certain predicted masks contained blurry edges or incomplete segmentation regions, particularly for smaller or more complex objects. These limitations are likely caused by the relatively shallow architecture, limited image resolution of 128×128 pixels, and the absence of advanced segmentation techniques such as skip connections or attention mechanisms.

Overall, the project successfully demonstrated how a CNN encoder-decoder architecture can be implemented from scratch for Salient Object Detection and how architectural improvements can positively affect segmentation performance.

6. Conclusion

This project presented an end-to-end implementation of a Salient Object Detection system using Convolutional Neural Networks built entirely from scratch with PyTorch. The project included dataset

preprocessing, augmentation, CNN architecture design, training, evaluation, visualization, and an interactive inference demo.

A baseline encoder-decoder CNN model was first implemented and evaluated using IoU, Precision, Recall, and F1-score metrics. An improved version of the architecture was then developed using Batch Normalization, Dropout regularization, and increased feature channels. The improved CNN achieved better segmentation performance and generated cleaner saliency masks compared to the baseline model.

The project provided practical experience with image segmentation, deep learning workflows, model evaluation, and computer vision applications. It also demonstrated the importance of architectural improvements, data augmentation, and loss function design in segmentation-based deep learning tasks.

7. Future Improvements

Several improvements could further enhance the performance of the Salient Object Detection system in future work.

Possible future improvements include:

- Using deeper segmentation architectures such as U-Net
- Adding skip connections to preserve spatial information
- Applying attention mechanisms for better feature localization
- Increasing image resolution for finer segmentation details
- Using pretrained backbone networks for stronger feature extraction
- Applying additional augmentation techniques to improve generalization

These improvements could potentially increase segmentation accuracy and produce more detailed and stable saliency masks.

References

[1] Wang, L., Lu, H., Wang, Y., Feng, M., Wang, D., Yin, B., & Ruan, X. (2017).

Learning to Detect Salient Objects with Image-level Supervision.

Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

[2] DUTS Dataset:

<https://www.kaggle.com/datasets/balraj98/duts-saliency-detection-dataset>

[3] PyTorch Documentation:

<https://pytorch.org/docs/stable/index.html>